

Learning to Balance a Pendulum on a Moving Board

Methods and results, and why each method mattered

We trained a small neural network, by reinforcement learning (RL), to keep a pendulum upright while its base moves — built up over three projects, each reusing the last.

Phase 1 · Flat ground — a robust balancer, deployed on hardware

- **Problem** — a classical controller (LQR) balanced in simulation but failed on the real motor: friction winds the arm into its cable limit.
- **Algorithm** — reinforcement learning with TQC (Truncated Quantile Critics), an off-policy actor-critic (SAC family) whose critic predicts a whole distribution of future rewards and drops the most optimistic ones to stay honest; exploration uses gSDE (state-dependent noise) for smoother, more transferable behaviour.
- **Smooth control** — CAPS: a reward penalty on abrupt command changes ($-0.02 \cdot (\Delta a)^2$) so the motor commands stay smooth and transfer to a real motor (jerky policies buzz/vibrate on hardware).
- **Deploy** — we run the trained policy (a tiny neural network, or MLP) directly on the microcontroller (ESP32), with no PC in the loop; removing that communication latency is what makes real-time balancing reliable.
- **Result** — full swing-up and balance on the real device.

Phase 2 · One tilting axis — add a sensor, keep the skill · verified 91.5%

- **Challenge** — the base tilts $\pm 20^\circ$ on one axis; the pole must stay upright to true gravity, not to the tilting base.
- **Sensor** — added a BNO086 IMU to read board tilt, and retargeted the reward to true gravity-vertical.
- **Reuse, don't restart** — instead of training a new controller from scratch for the harder task, we start from the already-working Phase-1 controller and fine-tune it (a 'warm start') — this saves training effort and keeps its proven skills.
- **Retention** — fine-tuning risks forgetting those skills, so during training we replay the old controller's successful experiences and gently pull the new actions toward the old ones. The critic learns faster than the actor — it must relearn the new task quickly, while the actor changes slowly so it doesn't wreck the proven policy. (gSDE off here — it caused an 'entropy collapse'.)
- **Verify** — 500 fresh trials per condition; success = stays balanced $\geq 80\%$ of the final 2 s (not a brief catch); reported with Wilson 95% confidence intervals (an error-bar on each rate that accounts for the finite number of trials).
- **Result** — 91.5% sustained success through one-axis tilt.

Phase 3 · Two tilting axes — diagnose and fix a hidden failure · current

- **Challenge** — the board now tilts in both roll and pitch, fast; capped at $\pm 15^\circ$ and $\sim 120^\circ/\text{s}$ to match the real moving board.
- **Failure** — the critic (a 'coach' that scores each situation) was out of its depth on the new motion, and because it scores itself (bootstrapping) the errors snowballed — rating clearly-good states as bad — so the controller got useless guidance and stalled.
- **Fix** — throw away the broken coach and start it fresh, then re-train it the easy way with the controller held still. With the controller fixed, the coach faces one stable question: from a given situation, how much reward does this fixed controller actually collect? We run the frozen controller in simulation and train the coach to match those real outcomes — a grounded task, free of the self-referential feedback that caused the blow-up — and only once its scores match reality do we let the controller improve again. We also shorten the planning horizon (a lower discount factor) for steadier scores and clip gradient sizes to prevent rare blow-ups.
- **Robustness** — we train several independent runs and keep the best; most reach the same strong envelope (Figure 3).
- **Result** — balances through fast two-axis board motion (see figures).

What made it work: build on the previous result instead of restarting · find the true cause cheaply before spending compute · trust only verified numbers.

Results

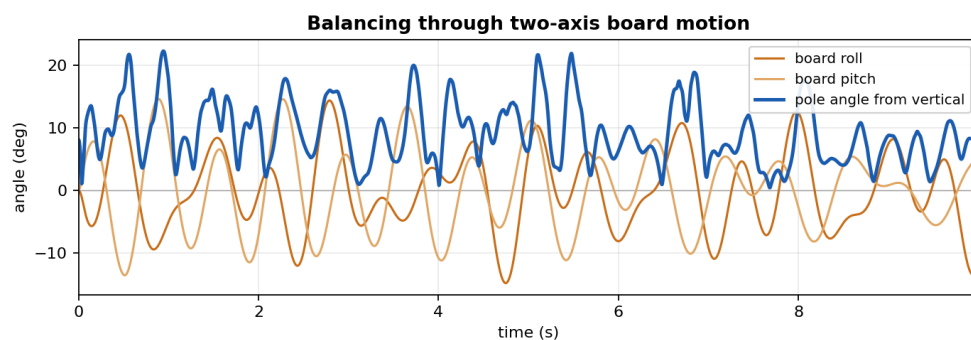


Figure 1. One episode: the board swings in roll and pitch (orange) while the pole stays near vertical (blue) — the task being solved.

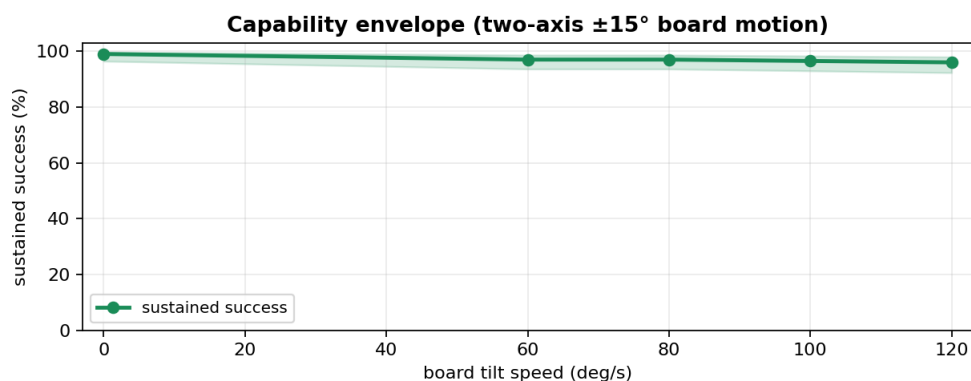


Figure 2. Capability (best controller): sustained success stays high (96–99%) across the whole speed range, up to fast two-axis motion (500-trial verification; shaded = 95% confidence).

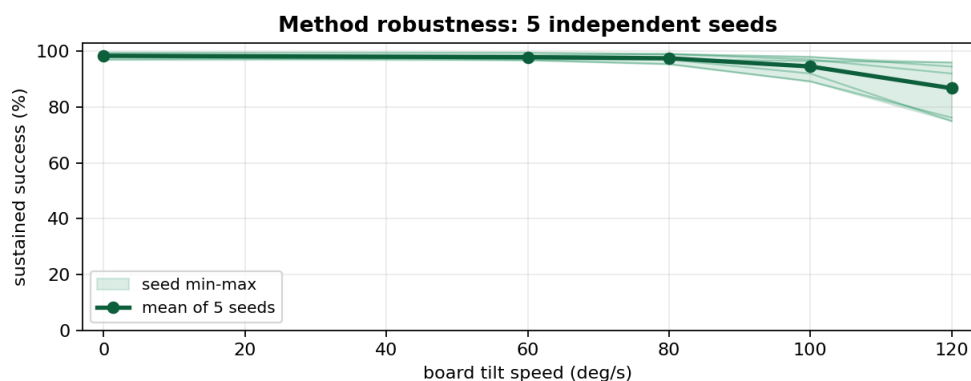


Figure 3. Robustness: five independently trained controllers are tightly matched up to $\sim 80^\circ/\text{s}$ and diverge only at the very fastest speed — the strong result is the method's, not luck.